

Intelligence Intent Router Design Specification — Implemented Edition

One chat, three modes, zero confusion. A deterministic router decides — in code — whether a message is conversation, a trade intent, or an order command. Trade intent triggers the advisory-first pipeline: analyze the market, check the protocol, deliver a verdict (favorable / unfavorable / wait), then stage a ticket. The model narrates; the router decides. Includes implementation guide, acceptance tests, and the addendum of what shipped in snapshot 4896a72.

DETERMINISTIC INTENT ROUTER

ADVISORY-FIRST TRADING

CONVERSATION STATE MACHINE

IMPLEMENTATION GUIDE

ACCEPTANCE TESTS

SHIPPED — SNAPSHOT 656528f

DEPLOYMENT & VERIFICATION

Contents

1 · The problem, in the user's own words	3
2 · Root cause: the model is currently the router	4
3 · The design: a deterministic intent router	5
4 · Advisory-first: the trade-intent pipeline	7
5 · The conversation state machine	9
6 · Implementation guide — files, functions, wiring	11
7 · Acceptance tests & edge cases	13
8 · As-built: shipped code, deployment & verification	15
Appendix A · What already shipped (mode lineage, snapshot 4896a72)	18
Appendix B · Checklist & questionnaire addendum for this feature	19

1 The problem, in the user's own words

The Intelligence chat cannot hold two thoughts at once. Today it behaves like this:

User: "buy Apple"

SYMPTOM 1

Intelligence: locks into trade mode — asks for quantity, stop, ticket strings. Every follow-up question ("wait — why that stop?", "actually what do you think of the market today?") is treated as more trade input. There is no way back to a normal conversation without abandoning the thread.

User: "what do you think about this market, should I be buying here?"

SYMPTOM 2

Intelligence: has a lovely conversation — and never touches the trade tools. The user who then says "okay, do it" often gets more conversation instead of a staged ticket.

The required behavior (the product owner's words, cleaned up): the default is intelligence — a real conversation. But when the message carries a trade trigger, the system must switch modes deliberately: before anything is staged, it **analyzes the market, checks against the protocol, and tells the user whether it's a good buy, a bad buy, or when to buy** — and only then moves toward a ticket. The switch between chatting, buying, and selling must be explicit and code-driven, not a vibe the model is having.

DESIGN PRINCIPLE (UNCHANGED HOUSE RULE)

The model narrates; code decides. Mode selection is a decision — therefore it moves from the prompt into the router, where it is deterministic, testable, and auditable. This is the same separation the rest of the platform already enforces: AI proposes, code disposes.

2 Root cause: the model is currently the router

The Intelligence runtime is well-built — live state snapshot, whitelisted tools, a single write tool (`proposeTrade`) that lands in the same confirmation gate as everything else. The defect is one layer up: **nothing in code decides what kind of message arrived**. The system prompt's Rule 2 ("when the user expresses a trade... extract... ask... only then call `proposeTrade`") is *advice to the model*, and like all prompt-layer rules it is probabilistic:

What should happen	What governs it today	Why it fails
Recognize "buy X" as trade intent	Model judgment under a 2,000-word system prompt	Over-trigger: trade framing hijacks the whole thread — the model keeps extracting parameters instead of conversing
Stay conversational by default	Model judgment	Under-trigger: in a conversational flow the model never reaches for the tool, even when the user says "do it"
Analyze before staging	Suggested, not sequenced	Nothing forces market analysis → protocol check → verdict <i>before</i> <code>proposeTrade</code> ; the model can jump straight to staging
One clean switch per message	None — the model re-decides every turn	Mode flickers mid-thread; context from one mode leaks into the other

THE DIAGNOSIS IN ONE SENTENCE

A load-bearing decision — *is this message conversation, trade intent, or an order command?* — is currently made by the least deterministic component in the system. Everything else in Requi that is load-bearing was moved into code for exactly this reason; the chat mode switch is the last one left in the prompt. This specification moves it.

Non-goals for this fix: no changes to the ticket pipeline, the confirmation gate, `proposeTrade`'s validation, the snapshot, or the tool whitelist — they are correct and stay. The change is a pre-classifier and a conversation state machine *in front* of the existing runtime, plus an advisory composer that runs the deterministic analysis before any staging is allowed.

3 The design: a deterministic intent router

Every incoming chat message passes through `classifyIntent()` — a pure function, no model call, no network — **before** the LLM is invoked. The router returns one of four modes, and the mode shapes everything downstream: which prompt addendum is used, which tools are offered, and whether staging is even possible this turn.

message → `classifyIntent()` → one of:

CHAT

default. conversation, questions, analysis, reviews.
proposeTrade is NOT offered this turn.

TRADE_INTENT

an instruction to transact ("buy ...", "sell ...", "flatten ...").
▶ enters the advisory-first pipeline (§4). `proposeTrade` is offered ONLY after the advisory has been delivered.

ORDER_COMMAND

"CONFIRM ORDER ..." / "REJECT ORDER ..." / "cancel ...".
▶ handled by the existing confirmation path, no advisory.

STATUS_QUERY

"what's open / P&L / positions / monitors / tickets".
▶ data tools only; read-only turn.

3.1 The trigger lexicon (deterministic, versioned, testable)

Class	Triggers (case-insensitive, word-boundary)	Notes
ORDER_COMMAND	confirm order · reject order · cancel order · cancel tkt-	Checked FIRST — these hit the existing confirmation path verbatim
TRADE_INTENT	buy · sell · long · short · flatten · close (my the) position · add to · exit	Must co-occur with a symbol token (or an open position, for exits) to qualify
STATUS_QUERY	p&l · positions · orders · tickets · monitors · what's open · how am i doing	Read-only tools only
CHAT	Everything else — including market opinions, "what do you think", strategy questions, post-trade reviews	

3.2 Negation & question guards — where routers usually break

A trigger word inside a **question** or a **conditional** is not an instruction — it's conversation ABOUT trading, and the correct mode is CHAT with a full analysis (the user is asking "good buy or bad buy?"). The guard is deliberately simple and conservative:

Message	Mode	Why
"buy Apple"	TRADE_INTENT	imperative + symbol

"buy 100 Apple"	TRADE_INTENT	imperative + quantity + symbol
"should I buy Apple?"	CHAT (+analysis)	question form — advise, don't stage
"is it a good time to buy?"	CHAT (+analysis)	question form
"what happens if I buy here?"	CHAT	hypothetical
"don't buy yet"	CHAT	negated imperative — acknowledge, stand down
"why did it buy?"	CHAT (post-trade review)	past tense — review, not instruction
"okay, do it — buy it"	TRADE_INTENT	imperative; "it" resolves to the symbol under discussion (\$5 context)
"CONFIRM ORDER TKT-12345"	ORDER_COMMAND	existing confirmation path

CONSERVATISM RULE

Ambiguous → CHAT. A missed trade intent costs the user one more sentence ("yes, buy it"); a false trade intent costs a staged ticket they never asked for. The router always errs toward conversation.

4 Advisory-first: the trade-intent pipeline

When the router returns `TRADE_INTENT`, the turn runs a fixed sequence **in code**. The model's only job in this mode is to narrate the result honestly. Nothing can be staged before the advisory exists — the tool list for this turn simply doesn't include `proposeTrade` until step 4 completes.

```
TRADE_INTENT(message)
|
| 1. EXTRACT      symbol · side · quantity? · stop?  (deterministic parse)
|   └─ symbol missing/ambiguous → ask, stay in ADVISORY
|
| 2. ANALYZE      marketDataService.track(symbol) → indicators
|   scanner.identifyStrategy(feed) → best-fit setup + confidence + reasons
|   data checks ($5 spec): fresh feed? enough bars? market open?
|   └─ stale/missing data → verdict UNFAVORABLE with the exact reason
|
| 3. PROTOCOL     governance package active? risk preview via sizePosition()
|   CHECK         portfolio heat gate · correlation gate (same functions the
|   └─ any gate blocks → verdict BLOCKED with the gate's verbatim reason
|
| 4. VERDICT      compose Advisory:
|   FAVORABLE     – setup identified, gates pass, data fresh
|   WAIT          – no setup yet; includes WHAT TO WATCH + when to re-ask
|   UNFAVORABLE   – data/quality reasons
|   BLOCKED       – a protocol gate forbids it (heat, correlation, governance)
|
| 5. NARRATE      model receives the Advisory object + snapshot and writes
|   the answer in plain language – verdict, evidence, proposed parameters,
|   and "say the word and I'll stage it"
|
| 6. STAGE        ONLY on a follow-up imperative ("do it", "stage it", "buy")
|   (opt-in)      does the next turn offer proposeTrade → ticket → CONFIRM
```

4.1 The Advisory object (the contract between code and narration)

```
interface Advisory {
  symbol: string; side: "BUY" | "SELL";
  verdict: "FAVORABLE" | "WAIT" | "UNFAVORABLE" | "BLOCKED";
  reasons: string[]; // verbatim engine evidence – trigger states, gate reasons
  dataQuality: { fresh: boolean; bars: number; marketOpen: boolean; delaySeconds: number };
  setup: { strategyId: string; confidence: number; state: string } | null;
  sizingPreview: { qty: number; dollarRisk: number; cappedBy: string } | null;
  proposed: { entry: number | null; stop: number | null; target: number | null } | null;
  watchFor: string | null; // WAIT verdicts: exactly what would change the answer
  governanceVersion: string;
}
```

4.2 What the user experiences

User: "buy Apple"

TRADE_INTENT

Intelligence: "Analysis before anything is staged — Apple is trading 0.8% above VWAP with RSI 61; the engine's best-fit setup is VWAP-reclaim but the hold-timer hasn't confirmed (6 of 10 minutes). Verdict: **WAIT** — a confirmed hold for 4 more minutes flips this to favorable. Proposed when it confirms: entry 214.30 stop 211.80 (risk 0.5% of equity, 200 shares). Say '**stage it**' and I'll put the ticket up, or ask me anything else first."

User: "actually what do you think of the overall market?"

CHAT — the switch works

Intelligence: normal conversation, full snapshot available, zero trade framing.

User: "okay stage it"

TRADE_INTENT (follow-up imperative)

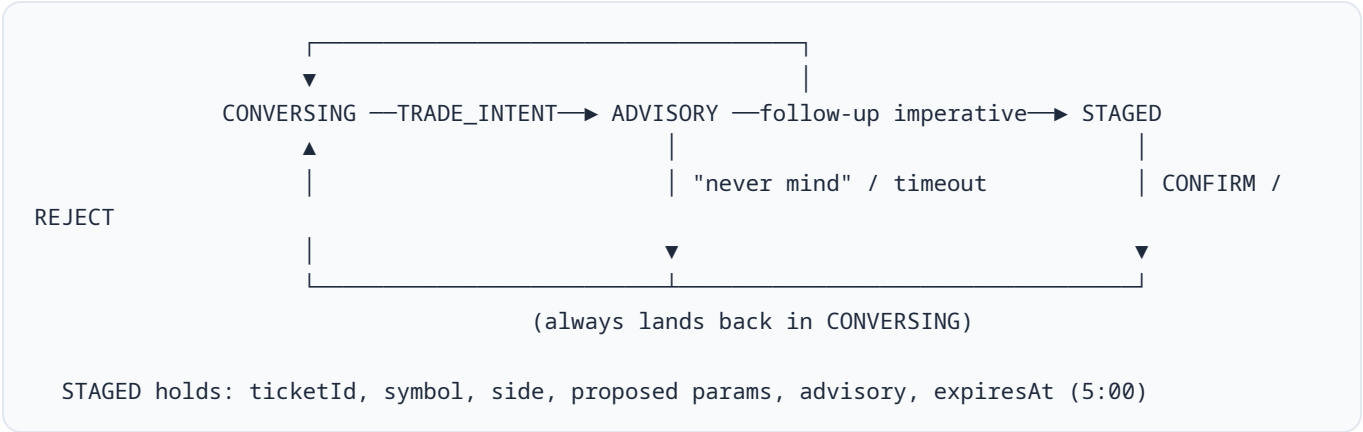
Intelligence: "Staged — ticket TKT-4471. Reply exactly **CONFIRM ORDER TKT-4471** to authorize, or **REJECT ORDER TKT-4471** to cancel. Nothing reaches a broker until you confirm."

WHY ADVISORY-FIRST IS THE INSTITUTIONAL PATTERN

It mirrors the risk-desk workflow the whole platform is modeled on: analysis → desk check → recommendation → execution authorization. It also kills the two symptoms at once: "buy X" no longer hijacks the thread (the advisory closes with an invitation, not an interrogation), and a conversational user who says "do it" gets a deterministic escalation path instead of hoping the model notices.

5 The conversation state machine

Mode is per-message from the router; **context** is per-thread from a small state machine. This is what makes "it", "stage it", and "cancel that" resolve correctly without asking the model to remember.



5.1 State contents & rules

State	What the thread remembers	Rules
CONVERSING (default)	last discussed symbol (if any)	proposeTrade never offered; all read tools available
ADVISORY	the Advisory object + symbol/side under discussion	"it" / "that" resolves to the advisory symbol; "stage/do it/buy it" escalates to STAGED; any new topic returns to CONVERSING
STAGED	ticketId + expiry	only ORDER_COMMAND or cancel language applies to the ticket; expiry (5:00) auto-returns to CONVERSING with a one-line notice

5.2 The clear switch, stated as invariants

- Default is CHAT.** Every thread starts CONVERSING; every completed or expired trade flow ends CONVERSING.
- One mode per message,** chosen by the router — never mid-turn, never by the model.
- Staging requires an imperative in TRADE_INTENT mode.** Questions, conditionals, and hypotheticals can never stage.
- Advisory precedes staging.** proposeTrade is physically absent from the tool list until an Advisory exists for that symbol+side in this thread.
- Timeouts are loud, not silent.** An expired staged ticket says so in one line and stands down.

PERSISTENCE

Thread state lives in the existing chat memory layer (per user, per thread, JSON blob beside the message history).
No new tables: `chatState = { mode, advisory?, stagedTicketId?, expiresAt? }` . On any read error the state defaults to CONVERSING — fail-safe toward conversation.

6 Implementation guide — files, functions, wiring

Sized for one focused engineer: one new module, two small edits to existing files, one prompt edit. Nothing in the ticket pipeline, confirmation gate, or tool executors changes.

6.1 New file: `api/intelligence/intent.ts` (~200 lines)

Export	Contract
<code>classifyIntent(message, threadState)</code>	Pure function → <code>{ mode: "CHAT" "TRADE_INTENT" "ORDER_COMMAND" "STATUS_QUERY", symbol?, side?, quantity?, reason } .Lexicon</code> + negation/question guards from §3. Word-boundary regex; symbol = 1–5 uppercase letters or a known tracked symbol; "it/that" resolves from <code>threadState.advisory</code> .
<code>TRADE_TRIGGERS / STATUS_TRIGGERS / COMMAND_TRIGGERS</code>	Exported constants — the versioned lexicon. Changing a trigger is a code edit with a test, never a prompt edit.
<code>composeAdvisory(userId, symbol, side)</code>	Runs §4 steps 2–4: track → indicators → identifyStrategy → governance status → sizePosition preview → portfolioHeatPct / returnCorrelation gates (imported from <code>api/engine/risk.ts</code> — the same functions the monitor uses). Returns the Advisory object; never throws on data failure — returns UNFAVORABLE with the reason (§5 spec never-estimate rule).

6.2 Edit: `api/intelligence/tools.ts` — the chat loop (three precise changes)

- Route before the loop.** First statement of the chat handler: `const intent = classifyIntent(message, threadState)` . Branch on `intent.mode` .
- Per-mode tool lists.** CHAT/STATUS_QUERY → all read tools, proposeTrade removed. ORDER_COMMAND → bypass the model entirely, straight to the existing confirmation path (exact-string handling already lives there). TRADE_INTENT → first call `composeAdvisory()` , inject the Advisory object into the snapshot, and offer proposeTrade *only* when the thread already holds a matching advisory (follow-up imperative) — otherwise the turn ends after narration.
- Thread state write-back.** Persist `{ mode, advisory?, stagedTicketId?, expiresAt? }` via `memory.ts` beside the message; on any read error default to CONVERSING.

6.3 Edit: `RUNTIME_ADDENDUM` — Rule 2 replaced

The prompt stops being the router. Rule 2 becomes narration discipline, not mode selection:

2. MODES ARE SET BY THE SERVER, NEVER BY YOU. Each turn arrives with a mode:

CHAT – converse normally; you have no staging tool this turn.

TRADE_INTENT – an ADVISORY object is in your snapshot. Narrate it faithfully: verdict, the engine's evidence (every reason string), proposed parameters, and close with "say 'stage it' and I'll put the ticket up". Never invent numbers beyond the object; never skip to staging language the object doesn't support.

STAGED — show the ticket id and the exact CONFIRM/REJECT strings. Say "staged", never "bought/sold/executed".

6.4 What is deliberately NOT touched

- `proposeTrade` executor and its validation (stop required, quantity bounds, origin INTELLIGENCE) — correct as-is.
- The confirmation path and exact-string governance check — `ORDER_COMMAND` routes to it unchanged.
- `buildSnapshot()` — the Advisory object is appended per-turn, the base snapshot is unchanged.
- Autonomous engine, monitors, trailing — this feature is Intelligence-module only.

EFFORT & RISK

~1 day including tests. Risk: LOW — additive layer in front of a stable runtime; worst-case fallback is a feature flag (`INTENT_ROUTER=off`) that restores current behavior by skipping the classifier and offering today's full tool list.

7 Acceptance tests & edge cases

The router is deterministic, so it is unit-testable without a model. These are the shipping gate — every row must pass before the feature flag defaults to on.

7.1 Router unit tests (no LLM involved)

Input (thread state)	Expected mode	Expected behavior
"buy Apple" (CONVERSING)	TRADE_INTENT	advisory composed; proposeTrade NOT offered; verdict narrated
"should I buy Apple?" (CONVERSING)	CHAT	analysis allowed via read tools; nothing staged; no ticket language
"don't buy yet" (ADVISORY)	CHAT	stands down; advisory cleared or parked; conversation continues
"stage it" (ADVISORY, symbol=AAPL)	TRADE_INTENT	"it" → AAPL; advisory already exists → proposeTrade offered → ticket staged
"stage it" (CONVERSING, no advisory)	TRADE_INTENT → advisory first	NO direct staging — advisory composed first (\$4 step order is enforced by tool availability)
"CONFIRM ORDER TKT-1" (any state)	ORDER_COMMAND	bypasses model; hits confirmation path; exact-string rule unchanged
"what's my p&l?" (any state)	STATUS_QUERY	read tools only; zero staging capability
"why did it buy?" (CONVERSING)	CHAT	post-trade review via getTickets/getMonitors; past tense never stages
"buy" (no symbol, CONVERSING)	TRADE_INTENT, symbol=null	asks which symbol; stays in advisory; nothing staged
"buy 100 Apple" mid-conversation about Tesla	TRADE_INTENT (AAPL)	explicit symbol beats thread context — the new advisory is for Apple

7.2 Advisory quality tests

1. **Data failure honesty.** Symbol with a dead feed → verdict UNFAVORABLE, reason = the feed's error string, sizingPreview null.
2. **Gate passthrough.** Heat at 9.8% + proposed 0.5% risk → verdict BLOCKED with the heat gate's verbatim reason — the same string the monitor would produce.
3. **Market closed.** dataQuality.marketOpen=false → verdict WAIT with "market opens 09:30 ET" in watchFor.
4. **No setup.** identifyStrategy returns nothing → WAIT with watchFor describing the missing condition in plain language.

7.3 Conversation-flow test (the owner's two symptoms, as scripted UAT)

1. "buy Apple" → advisory narrated, *then* "what do you think of the market?" → clean conversation with no trade framing. **Symptom 1 dead.**
2. Long conversation → "okay, do it" → deterministic escalation to advisory/staging without the model guessing. **Symptom 2 dead.**

DEFINITION OF DONE

7.1 all green · 7.2 all green · 7.3 both scripts pass on screen recording · no changes observed in the ticket pipeline's behavior (existing trades flow identically) · feature flag documented in the runbook.

8 As-built: shipped code, deployment & verification

Sections 1–7 were the design. This section is the record of what actually shipped in snapshot **656528f** (August 14, 2026), exactly how to deploy it, and exactly how to prove it works. Where the as-built differs from the §6 plan, the delta is called out and the reason given.

8.1 What shipped — three files

File	Change
<code>api/intelligence/intent.ts</code> NEW (~230 lines)	The deterministic router. <code>classifyIntent(text, thread)</code> → five modes: CHAT (default), TRADE_INTENT , ORDER_COMMAND , STATUS_QUERY , STRATEGIZE . Exported versioned lexicon (<code>TRADE_TRIGGERS</code> , <code>STATUS_TRIGGERS</code> , <code>STRATEGIZE_TRIGGERS</code> , <code>ORDER_COMMAND_RE</code> , <code>STAGE_FOLLOWUP_RE</code>) plus the question/negation guard. Thread state per user (<code>getThreadState</code> / <code>setThreadState</code> / <code>clearThreadState</code>). The Advisory contract and <code>composeAdvisory(userId, symbol, side)</code> — the 4-step pipeline: DATA (track/refresh; failure → UNFAVORABLE with the verbatim error, never estimated), ANALYZE (identifyStrategy setup + reasons), PROTOCOL (governance active/version + <i>the same</i> portfolioHeatPct / returnCorrelation functions the live monitor uses), VERDICT (FAVORABLE / WAIT / UNFAVORABLE / BLOCKED). Sizing preview only on FAVORABLE, and it is honest: the engine sizes from YOUR stop at staging, never an invented one.
<code>api/intelligence/tools.ts</code> EDIT	<code>agentChat(userId, text, opts?)</code> gains <code>AgentChatOptions : allowTradeTool</code> — when <code>false</code> , <code>proposeTrade</code> is <i>physically filtered out</i> of the tool list for that turn (the model cannot call what it cannot see); advisory — the engine's Advisory object is appended verbatim to the system message with narration discipline ("narrate faithfully, never invent or alter its numbers").
<code>api/intelligence-router.ts</code> EDIT	The chat endpoint wires the router behind the feature flag (<code>process.env.INTENT_ROUTER !== "off"</code>). Order: exposure guard → CONFIRM/REJECT ORDER (untouched) → classifyIntent → TRADE_INTENT : fresh imperative → <code>composeAdvisory</code> → narrate with the write tool removed (NO ticket this turn); imperative follow-up on a fresh advisory → the single staging turn (<code>allowTradeTool: true</code>) then the thread is cleared — fail-safe toward CONVERSING. STRATEGIZE → the existing strategy-builder branch. CHAT / STATUS_QUERY → <code>agentChat</code> with the write tool removed. If the model is unreachable, a deterministic fallback narrates the engine's verdict directly — the numbers still come from the engine.

8.2 As-built deltas from the §6 plan (and why)

Plan (§6)	As-built	Why
Thread state persisted via memory.ts blob	In-memory Map per process, 5-minute advisory TTL	Simpler and self-healing: a restart clears stale advisories; the TTL already bounded their life. Read errors cannot corrupt mode — the default is always CONVERSING.

Four modes	Five — STRATEGIZE added	"strategize / build me a strategy / backtest" now routes deterministically to the strategy-builder branch instead of relying on the model to notice.
Flag restores "current behavior"	<code>INTENT_ROUTER=off</code> restores the exact legacy path (direct trade-match staging), preserved verbatim in the else-branch	Rollback must be behavior-identical, not approximate.
Model narrates or nothing	Deterministic fallback narration when the model call fails	The engine's verdict is the product; the model is the voice. No voice ≠ no verdict.

8.3 Deployment steps (production)

- 1. **Snapshot / pull** the release at version **656528f**. Confirm the three files above are present and the working tree is clean.
- 2. **Environment:** `OPENAI_API_KEY` set (required for narration); `OPENAI_MODEL` optional (default gpt-5.6); **leave** `INTENT_ROUTER` **unset** — the router is ON by default. Set `INTENT_ROUTER=off` only to roll back.
- 3. **Build:** `rm -f *.tsbuildinfo && npm run build` → expect the Vite client build and `dist/boot.js` (~3.7 MB) with no type errors.
- 4. **Verify the bundle carries the router** (all four must print a non-zero count):
`grep -c classifyIntent dist/boot.js · grep -c composeAdvisory dist/boot.js · grep -c allowTradeTool dist/boot.js · grep -c INTENT_ROUTER dist/boot.js`
- 5. **Restart** production: stop the old process, then `NODE_ENV=production setsid nohup node dist/boot.js > /tmp/boot.log 2>&1 < /dev/null &`
- 6. **Boot log:** expect `[db] schema ensured` and `Server running` — no stack traces.
- 7. **Smoke contract:** root returns 200; signed-out `POST /api/trpc/intelligence.chat` returns **401** (a 404 means a stale deploy — rebuild). Note the mount path is `/api/trpc` .

8.4 Verification — the five-message proof script

Run signed-in, in order. This is the observable contract of the switch; every line is checkable from the chat UI plus the Tickets panel.

You say	Expected behavior	Proof it worked
"what do you think of the market today?"	CHAT — normal conversation via the AI model; no advisory, no trade language	Tickets panel: nothing created
"should I buy Apple?"	CHAT — question form; discussion allowed, nothing staged	Tickets panel: nothing created
"buy AAPL"	TRADE_INTENT — the deterministic engine composes an Advisory; the model narrates verdict + the engine's own reasons; no ticket this turn	Reply contains a verdict (FAVORABLE / WAIT / UNFAVORABLE / BLOCKED) with reason bullets; Tickets panel: still nothing

"stage it"	Follow-up on the fresh advisory → the single staging turn creates a ticket with a CONFIRM string	Ticket appears in Tickets panel / notification center, state CREATED; nothing submitted to any broker
"CONFIRM ORDER <id>"	ORDER_COMMAND — existing confirmation path, unchanged	Ticket submits; execution alerts follow as before

ROLLBACK CHECK

Set `INTENT_ROUTER=off` , restart, send "buy 10 AAPL" → a ticket stages immediately with no advisory (legacy behavior). Unset the variable, restart — the five-message script passes again. Both directions must work before sign-off.

8.5 Deployment checklist — sign-off

#	Confirmation item	Evidence
D1	Snapshot 656528f deployed; three changed files confirmed in the tree	Completed: _____ How: _____
D2	OPENAI_API_KEY present; INTENT_ROUTER unset (router ON by default)	Completed: _____ How: _____
D3	Clean build; all four bundle greps non-zero	Completed: _____ How: _____
D4	Boot log clean; root 200; signed-out chat returns 401 at /api/trpc mount	Completed: _____ How: _____
D5	Five-message proof script passes end-to-end (§8.4), Tickets panel verified at each step	Completed: _____ How: _____
D6	Rollback drill: INTENT_ROUTER=off restores legacy staging; router re-enabled afterward	Completed: _____ How: _____
D7	§7 acceptance suite green against the deployed build	Completed: _____ How: _____
D8	Regression: an existing paper trade still flows ticket → fill → position → P&L mode cards identically	Completed: _____ How: _____

A Appendix A · What already shipped (mode lineage, snapshot 4896a72)

Context for the same engineer: the previous issue — paper vs. live mode mixing — is **already fixed and deployed**. This section is the summary of what changed so it doesn't get re-litigated or accidentally reverted during the intent-router work.

Change	Where	Effect
<code>positions.broker</code> column	db/schema.ts + guarded ALTER	Every position carries its mode, copied from the source ticket at fill time; existing rows backfilled to PAPER
Mode stamped at fill	portfolio.ts recordFill	New positions get ticket.broker; scale-out children keep the ORIGINAL position's mode
Broker projected everywhere	listOrders, listPositions	Every order row and position row exposes <code>broker</code>
Mode-split P&L	getPnlSummary → byMode	<code>byMode.paper</code> / <code>byMode.live</code> — realized, unrealized, open/closed, W/L per mode; combined cards remain, labeled
UI badges + mode cards	OrdersPanel.tsx	PAPER (sky) / LIVE (teal) badges on orders and positions; two mode cards under the P&L strip

Full documentation: the *Autonomous Module Operations Guide* (21 pages) covers §1 paper-vs-live, §2 the ticket lineage, §4 the metrics dictionary, §5 a 22-item deployment checklist with completion notes, §6 the troubleshooting manual, §7 the 20-question accuracy questionnaire, and appendices with verification SQL and the exact diffs. **Read it before starting the intent-router work** — the intent router's `composeAdvisory` reuses the same risk-gate functions that guide documents.

DO NOT REGRESS

The intent router touches api/intelligence only. If any diff touches portfolio.ts, schema.ts, or the P&L projections, stop and re-read this appendix — those layers shipped and are under the Operations Guide's checklist, not this one.

B Appendix B · Checklist & questionnaire addendum (intent router)

Same discipline as the Operations Guide: every item gets a date/initials line and a "how completed" note.

B.1 Deployment checklist addendum

#	Item	Confirmation / notes
R1	<code>api/intelligence/intent.ts</code> exists; <code>classifyIntent</code> + <code>composeAdvisory</code> exported; <code>lexicon</code> constants exported	Completed: _____ How: _____
R2	Chat loop routes before the model; per-mode tool lists verified in code review	Completed: _____ How: _____
R3	ORDER_COMMAND bypasses the model and hits the existing confirmation path (diff shows no changes to that path)	Completed: _____ How: _____
R4	<code>proposeTrade</code> provably absent from the tool list in CHAT and STATUS_QUERY turns (log the per-turn tool list)	Completed: _____ How: _____
R5	Advisory-first enforced: staging physically impossible before an Advisory exists for that symbol+side in the thread	Completed: _____ How: _____
R6	Thread state persists via <code>memory.ts</code> ; read error → defaults to CONVERSING (test by corrupting the blob)	Completed: _____ How: _____
R7	Rule 2 in RUNTIME_ADDENDUM replaced with the narration-discipline text (§6.3)	Completed: _____ How: _____
R8	§7.1 router unit tests all green (paste the run output)	Completed: _____ How: _____
R9	§7.2 advisory quality tests all green	Completed: _____ How: _____
R10	§7.3 both symptom scripts pass — screen recording attached to the ticket	Completed: _____ How: _____
R11	Feature flag INTENT_ROUTER documented; off = current behavior restored	Completed: _____ How: _____
R12	Regression sweep: an existing paper trade still flows ticket → fill → position → P&L mode cards identically (Appendix A)	Completed: _____ How: _____

B.2 Questionnaire addendum (expected answers printed)

1. **Who decides the mode of a chat message?** — *Expected: the deterministic router (`classifyIntent`), never the model.*
2. **What is the default mode, and where does every trade flow end?** — *Expected: CHAT / CONVERSING; every completed or expired flow returns to CONVERSING.*

3. **"should I buy Apple?" stages a ticket. True or false, and why?** — *Expected: false — question form is CHAT; staging requires an imperative in TRADE_INTENT mode, and an advisory must exist first.*
4. **What must exist in the thread before proposeTrade is even offered?** — *Expected: a matching Advisory object (symbol+side) composed this thread.*
5. **Which two risk functions does composeAdvisory share with the live monitor, and why does that matter?** — *Expected: portfolioHeatPct and returnCorrelation from engine/risk.ts — one implementation means the advisory's BLOCKED verdicts match what the monitor would do; no drift between chat advice and engine behavior.*
6. **The classifier fails to read thread state. What mode results, and is that safe?** — *Expected: CONVERSING — fail-safe toward conversation; nothing can be staged.*